Grade 11 C++ Project Report Template

Written Project Report - 50 Points

Student Name: Amogh Sheth and Samaksh Bhargav	Project Title: Personal movie rating system
Date:11/06/2026	Class: Grade 11 C++

1. Project Goal - 5 points

Explain what you wanted your C++ program to accomplish.

Our main goal behind this C++ project was to develop a personal movie rating system that allows users to organize, rate, and keep track of movies they have watched or plan to watch. We hoped to create a program that was both practical and easy to use, while also, at the same time, demonstrating the C++ programming concepts we learned. Users can add movies by entering information such as the title, genre, release year, rating, and an optional written review. Then, the program stores this information using objects and allows users to view, search, edit, or remove movies from their collection.

One of the main features we had in mind and wanted to implement was an automatic recommendation system. Based on a movie's rating, the program determines whether it is recommended, not recommended, or has not yet been rated. We also wanted the program to calculate useful statistics, including the average movie rating, the number of recommended movies, and the number of movies on the user's watchlist. By combining these features, our goal was to create a functional movie management system that makes it easier for users to organize their collection and reflect on their movie preferences.

2. Purpose of the Project - 5 points

Explain why you created this program. Include what problem it solves, who could use it, and why you chose this project.

We created our personal movie rating system to solve the common problem that many movie enthusiasts (such as us) experience: keeping track of the movies they have watched, how much they enjoyed them, and what they want to watch next. Although people often remember their favorite movies, it can be difficult to recall specific ratings, opinions, or recommendations as time progresses throughout their life. Our program addresses this issue by providing one singular place where users can store movie information, write reviews, manage a watchlist, and easily access their previous ratings.

The program is designed for anyone who enjoys watching movies, whether they are casual viewers, students, or dedicated movie enthusiasts. For example, someone who watches several movies each month could use the system to keep track of their experiences and compare their ratings. Similarly, someone looking for a movie to watch could review their collection and use the recommendation statuses to help make a decision.

We chose this project because movies are something we are both genuinely passionate about, which made the development process more engaging and meaningful, as it is actually something that we could see ourselves using. Rather than creating a program that only demonstrated programming concepts, we wanted to build something connected to our personal interests that could also be useful in everyday life. A movie rating system was suitable for C++ specifically because it allowed us to apply classes, objects, conditional statements, loops, functions, and mathematical calculations in a real-world scenario. For instance, we used a rating threshold of 7.0 to determine whether a movie should be recommended. This gave us an opportunity to turn numerical data

into meaningful results while strengthening our understanding of how programming can solve real-world problems.

3. Program Description - 8 points

Describe how your program works from beginning to end. Explain the input, processing, and output/results.

Our program begins by creating a Catalog object that stores and manages a collection of movies. It can initially load nine sample movies, such as Interstellar and Spirited Away, with each movie created using the parameterized constructor in the Movie class.

Input: The user interacts with the program through a console menu, where they can view, add, edit, delete, or search for movies, check statistics, or exit. When adding or editing a movie, the user enters its title, genre, release year, rating, and an optional review. Ratings range from 1.0 to 10.0, while a rating of 0 places the movie on the watchlist.

Processing: The program first validates the information to ensure that titles are not empty, genres are accepted, years fall between 1888 and 2100, and ratings are valid. It then stores the information in Movie objects within a vector. Using if/else statements, the program classifies movies as "Recommended" for ratings of 7.0 or higher, "Not recommended" for lower ratings, or "Not rated" for a rating of 0. Loops are also used to search the collection and calculate statistics, including the average rating, which excludes unrated movies.

Output: The program displays each movie's information, including its ID, title, genre, year, rating, recommendation status, and review. It also provides statistics such as the total number of movies, average rating, recommended count, highest-rated movie, and genre distribution. For example, the seven rated sample movies have a combined score of 59.7, giving an average of approximately 8.53, with six movies recommended and two additional movies on the watchlist.

4. C++ Concepts Used - 8 points

Complete the table for the concepts actually used in your project.

C++ Concept	**How I Used It in My Project**
Variables / Data Types	We used several data types to store and manage movie information. The **int** data type stores movie IDs, release years, and various counts, while double stores numerical ratings and calculated averages. We used **string** to represent movie titles, genres, and written reviews. Additionally, **bool** values help indicate whether certain operations, such as adding or updating a movie, were successful. *Choosing the appropriate data type for each piece of information helped keep our program organized and reliable.*
cin / cout	We used **cout** to display the main menu, instructions, movie information, statistics, and error messages in the console. To receive information from the user, we used C++ input operations, including **getline(cin, ...)**, which allows the program to read complete lines of text. This was especially important for movie titles and reviews that might contain spaces.

	These input and output operations make it possible for users to interact directly with our program.
if / else	Conditional statements play an important role in validating information and making decisions. For example, the **Movie::status()** method uses **if/else** statements to determine whether a movie is recommended, not recommended, or unrated. We also use conditional statements to check whether the release year is between 1888 and 2100, whether ratings are valid, and which menu option the user selected. *This allows the program to respond appropriately to different inputs and situations.*
Loops (if used)	We used a **while** loop to continuously display the menu until the user chooses to exit the program. We also used **for** loops to move through the collection of movie objects when displaying movies, searching for specific entries, and calculating statistics. For example, a loop adds together the ratings of all rated movies to calculate the average. *Loops make these operations more efficient because the same instructions can be repeated without writing separate code for every movie.*
Classes and Objects	Our program uses two main classes, **Movie** and **Catalog**, to organize its functionality. The **Movie** class represents an individual film and stores its information, while the **Catalog** class manages the collection of movies and the operations performed on them. Each movie is represented by its own object, allowing multiple films to be stored independently. The program begins with nine movie objects, demonstrating how classes can be used to create and manage related data.
Methods / Functions	We used methods to divide the program into smaller, more manageable tasks. For instance, **status()** determines a movie's recommendation, while methods such as **add()**, **update()**, **remove()**, **query()**, and **stats()** handle different catalog operations. We also created helper functions to process numerical and text input. *Organizing our code into functions made it easier to understand, test, debug, and modify without affecting unrelated parts of the program.*
Constructor	We used a parameterized constructor in the **Movie** class to initialize each movie object with its specific information. The constructor, **Movie(int id, string title, string genre, int year, double rating, string review)**, assigns the values passed into it to the object's private data members.

	This ensures that every movie is created with the information needed for the program to manage it correctly.
Other	We applied **encapsulation** by making the data members of our classes **private** and using **public** methods to interact with them. This helps protect movie information from being modified directly by unrelated parts of the program. We also used a vector to store multiple Movie objects in a collection that can grow or shrink as movies are added or removed. Finally, we performed mathematical operations involving sums, averages, counts, and maximum values to generate meaningful statistics about the user's movie collection.

5. How I Built the Program - 7 points

Describe the main steps you followed to turn your idea into a working C++ program.

We began by choosing a personal movie rating system because we both enjoy movies and wanted to create something practical using the C++ concepts we learned. We identified the information that each movie would need, including its title, genre, release year, rating, and review. We also agreed upon a recommendation rule where movies rated 7.0 or higher would be considered recommended.

Next, we designed two classes: Movie and Catalog. The Movie class stores each film's information using private variables, a constructor, and methods, while the Catalog class manages the collection using a vector. We then developed a console menu using cin, cout, and loops, allowing users to add, edit, delete, search, and view movies and statistics.

Once the main features were working, we connected our C++ classes to a browser-based interface to make the program easier to use. Finally, we tested different inputs, checked the recommendation threshold, and verified that our calculations were accurate. We fixed any errors we encountered to ensure the program worked reliably.

6. Troubleshooting and Debugging - 8 points

Describe at least two problems or errors you experienced and how you fixed them.

Problem / Error	**What Caused It?**	**How I Fixed It**
Movie titles were sometimes skipped or recorded as empty after selecting a menu option.	This problem occurred because we were mixing **cin >>** with **getline()** when reading user input. When cin >> reads a number, it can leave a newline character in the input buffer. As a result, the next getline() call may read that leftover newline instead of waiting for the user to enter a movie title.	We solved this issue by using **getline()** to read complete lines of input and then converting numerical entries using a string stream. This prevented leftover newline characters from interfering with later prompts and ensured that users could enter movie titles correctly, including titles containing spaces.

Some valid decimal ratings, such as 8.1, were incorrectly rejected by the program.	The issue was caused by floating-point precision. Because computers represent many decimal numbers using binary floating-point values, certain decimals cannot be stored with perfect accuracy. Our original validation logic checked whether ratings were exact multiples of 0.1, which occasionally caused valid values to fail the test.	We adjusted our rating validation to allow a very small tolerance when checking decimal increments. This accounts for minor floating-point inaccuracies while continuing to reject ratings outside the accepted range of 1.0 to 10.0, except for 0, which represents an unrated movie. The change allowed valid ratings such as 8.1 to be accepted consistently without weakening our overall input validation.

What did troubleshooting teach you about programming?

Troubleshooting taught us that programming requires patience, logical thinking, and close attention to detail. Throughout the project, we realized that even seemingly small issues, such as leftover input characters or floating-point rounding, can cause unexpected errors and even crash the whole program. We also learned the importance of testing different situations, including invalid inputs, ratings near the recommendation threshold, and cases where no movies have been rated. These challenges showed us why proper input validation and error handling are essential for creating a reliable program. Most importantly, we learned to approach debugging **systematically** by identifying the problem, examining the relevant code, making adjustments, and testing again.

7. Testing the Program - 4 points

Provide at least two test cases.

Test	Input	Expected Result	Actual Result
Test 1: Adding a movie below the recommendation threshold	We added a movie titled "Class Demo" with the genre Drama, release year 2026, and a rating of 6.9.	The program should accept the valid information, create a new movie entry, and display the movie in the catalog. Because the rating is below 7.0, its recommendation status should be "Not recommended."	The program successfully saved "Class Demo" in the catalog and displayed the correct movie information. Its status was automatically set to "Not recommended," confirming that the rating comparison worked correctly.
Test 2: Updating a movie to meet the recommendation threshold	We edited the previously added movie, "Class Demo," and changed its rating from 6.9 to 7.0.	The program should update the movie's rating and change its status to "Recommended." The total number of recommended movies should increase by one, and the average rating should be recalculated	The program successfully updated the movie's rating and changed its status to "Recommended." The recommended movie count increased as expected, and the statistics and average

		using only movies with valid ratings.	rating were recalculated correctly.

Testing Explanation:

These tests were particularly important because they allowed us to verify that the recommendation system worked correctly at its exact rating boundary. By comparing ratings of 6.9 and 7.0, we confirmed that the program did not recommend movies below the required score and correctly identified movies that met the threshold. The second test also demonstrated that editing an existing movie updated the related statistics rather than just simply changing the displayed rating.

8. Final Reflection - 5 points

What did you learn? What was most challenging? What are you most proud of? What would you improve or add?

Throughout this project, we developed a stronger understanding of how C++ concepts work together to create a functional program. We learned how classes and objects organize information, how constructors and methods manage data, and how loops and conditional statements can process information and make decisions. Applying these concepts to a real project helped us understand their purpose beyond classroom examples.

The most challenging part was handling user input and ensuring our calculations were accurate. We encountered issues involving leftover input characters, decimal rating precision, and excluding unrated movies from the average. Solving these problems taught us the importance of careful testing, debugging, and attention to detail.

We are most proud of successfully creating a system where the Movie and Catalog classes work together to manage multiple movies, recommendations, and statistics. We also exceeded the three-object requirement by including nine sample movies. In the future, we would improve the program by adding permanent file storage, better search functionality, and the ability for multiple users to manage their own collections.

Student Submission Checklist

☐ Written project report completed	☐ C++ program completed
☐ Video includes goal and purpose	☐ Video demonstrates the program
☐ Video explains how the program was made	☐ Video explains troubleshooting/debugging
☐ Final reflection included	☐ File/video uploaded correctly